

What is a JOIN?

A Simple Explanation Without Any Code

Imagine you have two lists. You want to combine them. That is all a JOIN does. It takes two lists and combines them based on something they have in common. Let us look at real-life examples before we touch any SQL.

We will use three everyday situations. By the end, you will understand INNER JOIN, LEFT JOIN, and why NULL values appear. Then we will connect everything back to SQL.

Analogy	What It Explains
1. Phone + Facebook	INNER JOIN vs LEFT JOIN
2. Fruits + Prices	What happens to items with no match
3. Finding an address	Multi-table JOIN chain

Three analogies. Three levels of understanding.

Analogy 1: Phone Contacts vs Facebook Friends

You have a phone with 5 contacts. Some of them are also your Facebook friends, some are not. Think about these two questions:

List A: Your Phone Contacts

Name	Phone Number
Ali	055-111
Aysel	055-222
Kamil	055-333
Nargiz	055-444
Rashad	055-555

List B: Your Facebook Friends

Name	Profile Picture
Ali	beach.jpg
Aysel	selfie.jpg
Samir	cat.jpg
Leman	sunset.jpg
Rashad	car.jpg

Notice: Ali, Aysel, and Rashad appear in BOTH lists. Kamil and Nargiz are only in your phone. Samir and Leman are only on Facebook.

Question 1: "Who is in BOTH lists?"

You cross-reference the two lists by Name. If a name appears in both lists, you keep it. If a name is only in one list, you throw it away.

Name	Phone Number	Profile Picture
Ali	055-111	beach.jpg
Aysel	055-222	selfie.jpg
Rashad	055-555	car.jpg

3 people appear in BOTH lists. Kamil, Nargiz, Samir, Leman are gone.

This is INNER JOIN. It only keeps the rows that match on BOTH sides. If there is no match, the row disappears completely.

Question 2: "Show me ALL my phone contacts, and add their Facebook photo if they have one"

This time, you start with your phone contacts (List A). For each contact, you check if they are on Facebook. If yes, you add their photo. If no, you still keep the contact but write "no photo" for the picture.

Name	Phone Number	Profile Picture
Ali	055-111	beach.jpg
Aysel	055-222	selfie.jpg
Kamil	055-333	---
Nargiz	055-444	---
Rashad	055-555	car.jpg

All 5 phone contacts are here. Kamil and Nargiz have "---" instead of a photo.

This is LEFT JOIN. It keeps ALL rows from the left list (phone contacts). If there is no match in the right list (Facebook), it fills in NULL (shown as "---" here). The row does NOT disappear.

Analogy 2: Shopping List vs Store Prices

You go to a fruit market with a shopping list. You want to check the price of each item on your list. Some items are available, some are not in season today.

List A: Your Shopping List

Item	Quantity
Apple	2 kg
Banana	3 kg
Cherry	1 kg
Grape	2 kg
Fig	500 g

List B: Today's Prices

Item	Price per kg
Apple	\$3.00
Banana	\$2.00
Mango	\$8.00
Watermelon	\$5.00
Grape	\$4.00

Apple, Banana, and Grape appear in both lists. Cherry and Fig are on your list but the store does not have them today. Mango and Watermelon are in the store but not on your list.

You ask: "What is the price for everything on MY list?"

Item	Quantity	Price per kg	Total Cost
Apple	2 kg	\$3.00	\$6.00
Banana	3 kg	\$2.00	\$6.00
Cherry	1 kg	---	???
Grape	2 kg	\$4.00	\$8.00
Fig	500 g	---	???

Cherry and Fig: no price available. Shown as "---" (NULL in SQL). But they are NOT removed from your list. You still know you want them.

Now compare with INNER JOIN:

If you used INNER JOIN, you would ONLY see items that the store has today. Cherry and Fig would be completely removed from the result. You would not even know they were on your original list.

Item	Quantity	Price per kg	Total Cost
Apple	2 kg	\$3.00	\$6.00
Banana	3 kg	\$2.00	\$6.00
Cherry	1 kg	---	???
Grape	2 kg	\$4.00	\$8.00
Fig	500 g	---	???

With INNER JOIN: Cherry and Fig are GONE. You lost information about what you wanted.

Key Insight: LEFT JOIN preserves your original list. INNER JOIN shrinks it. In data analysis, you almost always want LEFT JOIN because losing data silently is dangerous. You want to SEE what did not match (the NULL values) so you can investigate why.

Analogy 3: Finding a Friend's Full Address

This is how multi-table JOINS work. Imagine you want to find your friend's full address. You cannot find it in one place. You have to look it up step by step.

Step 1: You know your friend's name

You start with the name: Ali. You also know his apartment number. But you want the full address with city and country.

Friend	Apartment No.
Ali	Apt 15

This is the starting point. Like FROM employees.

Step 2: Look up which building this apartment is in

You check the apartment directory. Apt 15 is in Building B. Now you have: Ali, Apt 15, Building B.

Friend	Apartment No.	Building
Ali	Apt 15	Building B

Like JOIN departments. New column added!

Step 3: Look up which street this building is on

Now you use Building B (from Step 2 result, NOT from some other directory) to find the street. Building B is on Nizami Street.

Friend	Apartment	Building	Street
Ali	Apt 15	Building B	Nizami St.

Like JOIN locations. Used "Building B" from the PREVIOUS result!

Step 4: Look up which city this street is in

You use Nizami Street (from Step 3) to find the city: Baku.

Friend	Apartment	Building	Street	City
Ali	Apt 15	Building B	Nizami St.	Baku

Step 5: Look up which country this city is in

You use Baku (from Step 4) to find the country: Azerbaijan.

Friend	Apt	Bldg	Street	City	Country
Ali	15	B	Nizami	Baku	Azerbaijan

Each step adds one piece of information. You built the full address step by step.

The chain rule: In Step 3, you used "Building B" from Step 2. You did NOT go back to the original apartment directory. You used the information you ALREADY FOUND. This is exactly how multi-table SQL JOINS work. Each JOIN uses the result built by all previous JOINS.

What If the Chain Breaks?

Now imagine your friend Aysel. She lives in Apt 99. You look up Apt 99 in the directory, but it is not listed. The directory is old and does not have her apartment yet.

Step	What You Do	Result for Aysel
Start	Name + Apt	Aysel, Apt 99
Step 2	Look up Apt 99 in building directory	Not found! (NULL)
Step 3	Look up building... which building?	Cannot! NULL building
Step 4	Look up street...	Cannot! NULL street
Step 5	Look up city...	Cannot! NULL city

At Step 2, the apartment directory does not have Apt 99. So Aysel has NULL building. Because she has no building, Step 3 cannot find a street. NULL street. Step 4 cannot find a city. NULL city. Step 5 cannot find a country. NULL country.

But with LEFT JOIN, Aysel is still in the result. She just has NULLs for everything that depends on the missing building. If this were INNER JOIN, Aysel would completely disappear.

LEFT JOIN vs INNER JOIN for Both Friends

Friend	Apt	Building	Street	City	Country
Ali	15	B	Nizami	Baku	Azerbaijan
Aysel	99	---	---	---	---

LEFT JOIN: Both friends are here. Aysel has NULLs but she exists.

Friend	Apt	Building	Street	City	Country
Ali	15	B	Nizami	Baku	Azerbaijan
Aysel	99	---	---	---	---

INNER JOIN: Aysel is GONE. You lost a friend!

Now Let Us Map This Back to SQL

Every analogy we used has a direct mapping to SQL concepts. Here is the complete picture:

The Word Mapping

Everyday Word	SQL Word	Meaning
List	Table	A collection of data rows and columns
Item on the list	Row	One record in the table
Property of the item	Column	One piece of data (name, price, etc.)
Match by name	ON name = name	The condition that connects two tables
Cross-reference two lists	JOIN	Combine rows from two tables
Keep only matches	INNER JOIN	Remove rows with no match on either side
Keep everything from left	LEFT JOIN	Keep all left rows, NULL if no match on right
"Not found" / "no photo"	NULL	Missing value - the data does not exist
Chain of lookups	Multi-table JOIN	Each JOIN uses the result of previous ones
NULL breaks the chain	NULL propagation	If one step is NULL, dependent steps are also NULL

The Analogy Mapping

Analogy	LEFT =	INNER =
Phone + Facebook	All contacts + photo if exists	Only contacts with Facebook
Shopping + Prices	All items + price if available	Only items in stock today
Address chain	All people + address if found	Only people with full address
SQL employees	All employees + dept if assigned	Only employees with a department

Summary: The 5 Golden Rules of JOINS

1. JOIN = Combining Two Lists

A JOIN takes two tables (lists) and combines them based on a common value. Like matching your phone contacts with your Facebook friends by name.

2. INNER JOIN = Only the Matches

If a row does not have a match in the other table, it is removed completely. Like only keeping phone contacts who are also on Facebook.

3. LEFT JOIN = Keep Everything + Add What Matches

ALL rows from the left table are kept. If there is no match on the right, NULL is filled in. Like keeping all phone contacts and adding their photo if they have one.

4. NULL Breaks the Chain

If one JOIN produces NULL, every subsequent JOIN that depends on that NULL will also produce NULL. Like not finding the building and therefore not finding the street, city, or country. But LEFT JOIN still keeps the row.

5. Each JOIN Uses the Previous Result

In a chain of JOINS, each new JOIN works on the combined result of all previous JOINS. Like finding an address: you use the building from Step 2 when looking up the street in Step 3. You do not go back to the original apartment directory.

INNER JOIN = "Only give me the matches"

LEFT JOIN = "Give me everything from my list, and add info if available"